



MattRM2 VFX Build

Release 1.1.0

Github-> https://github.com/MattRM2/MattRM2_VFX_Build.git

CYCLES · DEEP EXR · VFX PIPELINE

MattRM2 VFX Build

Documentation — based on Blender 5.2.2 LTS

A custom Blender build for professional VFX pipelines. This document covers the production features added on top of official Blender 5.2.2 LTS, plus the included bug fixes.

Created by Matthieu Barbié · 2026

Release 1.1

Unofficial build — not affiliated with or endorsed by the Blender Foundation.

0

Table of Contents

1

Deep EXR — Cycles (CPU + GPU)

- 1.1 Overview
- 1.2 Panel Reference — View Layer > Passes > Deep EXR
- 1.3 Quick Start
- 1.4 Output Structure
- 1.5 GPU Rendering & Volume Modes
- 1.6 Multi-Layer Deep EXR Output

2

Z-Depth Pass — Antialiased, Volume & Transparency Aware

- 2.1 Overview
- 2.2 What the Pass Does
- 2.3 Limitations

3

GPU Render Options

- 3.1 Render Devices in the Device Panel
- 3.2 OSL Group-Data Budget
- 3.3 Trade-off & Constraints
- 3.4 Choosing a Backend

4

Environment Variables in File Paths

- 4.1 Overview & Supported Syntaxes

5

Qt Tools (PySide6)

- 5.1 Overview
- 5.2 Enabling the Deep EXR Demo
- 5.3 Building a Tool — for Developers

6

View Layer Attribute Overrides

- 6.1 Overview & Workflow
- 6.2 Per-Layer Render Engine
- 6.3 Per-Layer Samples & Shaders
- 6.4 Scope & Limitations

7

Curves Lofting

- 7.1 Inputs and Options
- 7.2 Lofting a Collection
- 7.3 Outputs
- 7.4 Rules

8 Included Bug Fixes

- 8.1 Node Editor Click-Drag
- 8.2 Volume Indirect-Only Shadows
- 8.3 Texture Cache with CPU + GPU Rendering
- 8.4 Blend File Icon on Windows

9 Known Limitations & Roadmap

- 9.1 Known Limitations
- 9.2 Roadmap

1 Deep EXR — Cycles (CPU + GPU)

Industry-standard deep compositing, Arnold / RenderMan architecture

This build adds a full **Deep EXR** implementation to Cycles, following the **Arnold / RenderMan architecture**, on **CPU, CUDA and OptiX (SVM + OSL)**. Deep EXR is the industry-standard format for high-quality depth compositing — it stores multiple colour and depth samples per pixel, which lets a compositor merge elements by true scene depth instead of a flat 2D over.

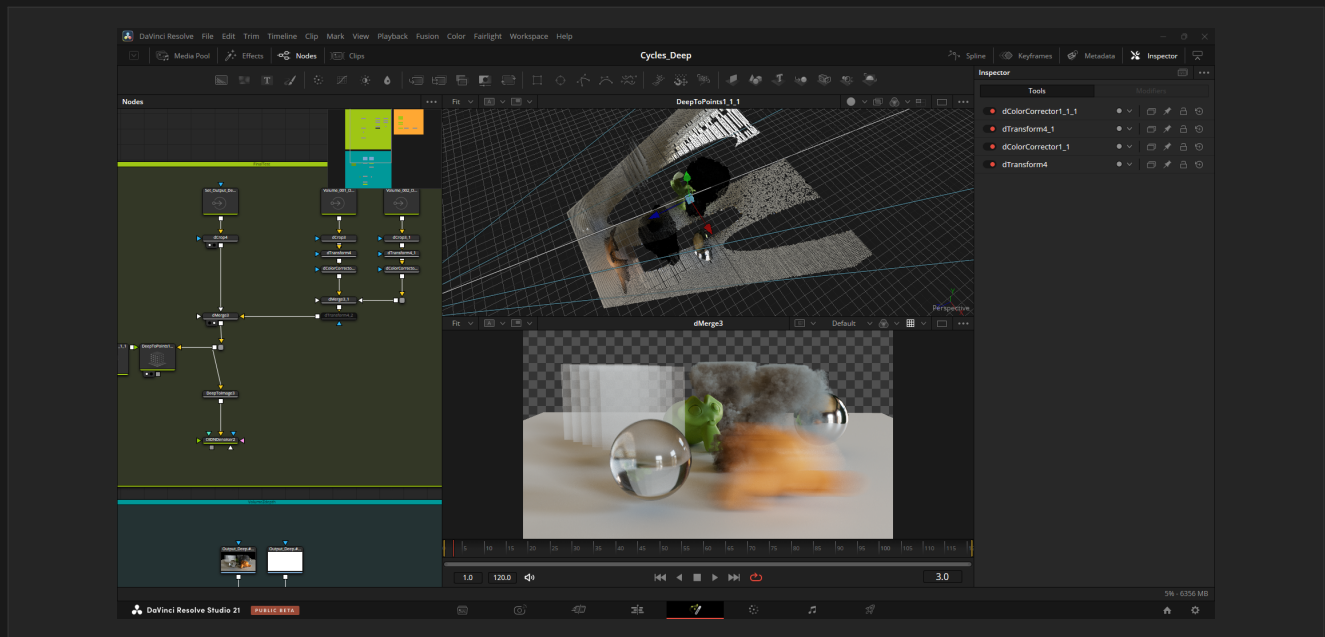


Fig. 1.1 — Cycles Deep EXR composited in DaVinci Resolve — Fusion

1.1 Overview

Key characteristics:

- **Surfaces** — recorded per sample, so Motion Blur and Depth of Field are preserved naturally. The deep flatten is pixel-perfect identical to the standard flat EXR render.
- **Volumes** — Arnold-style raymarch, decoupled from the path-traced volume integration, with **Depth of Field and Motion Blur** support. Multiple overlapping/intersecting volumes on the same view layer are fully supported.
- **Shadow catcher** — the catcher is recorded in the deep as a **shadow matte** (black RGB, shadow-density alpha), ready to deep-merge over footage — including catchers seen behind or through volumes.
- **Output** — standard OpenEXR deep scanline format, readable by Nuke, Fusion, Resolve, and any OpenEXR-compliant compositor. Single-layer deep files carry the view layer name as their part name.
- **Per-view-layer output** — each view layer gets its own subfolder and filename prefix automatically.

Pixel-perfect flatten

The deep flatten matches the beauty render exactly: you can drop the deep EXR into your comp and get the same image as the flat render, then gain depth-based merging on top.

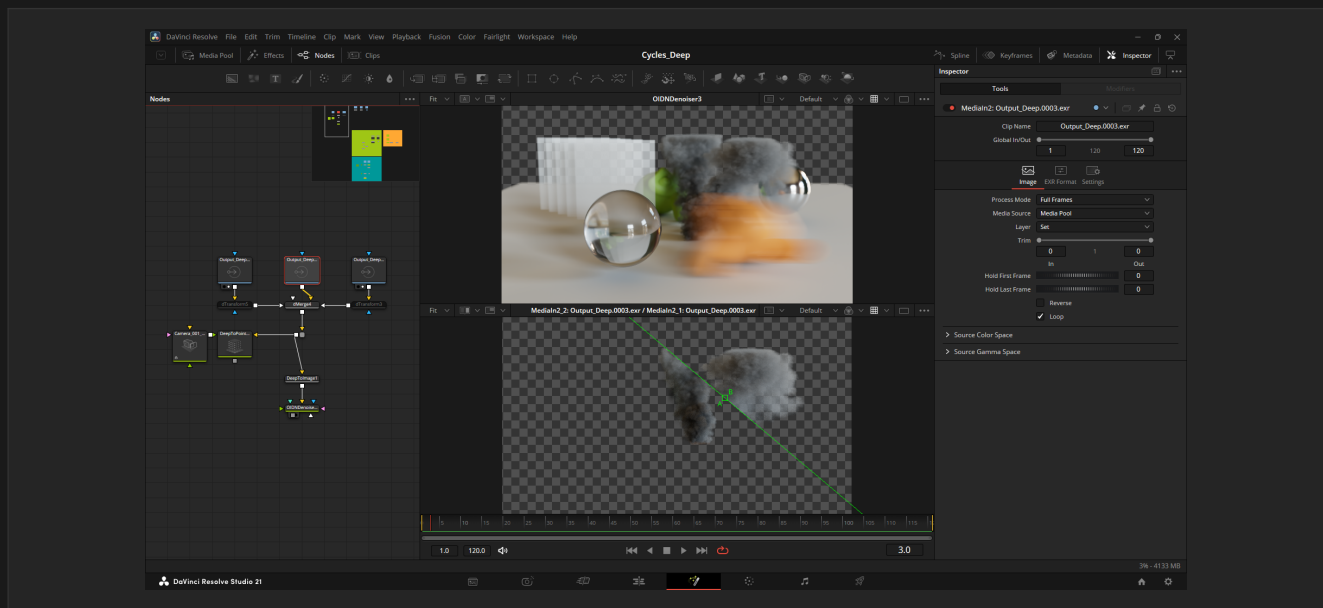


Fig. 1.2 — Deep volumes with Depth of Field, Motion Blur and shadows

1.2 Panel Reference — View Layer > Passes > Deep EXR

All Deep EXR settings live in **View Layer Properties > Passes > Deep EXR**. Each view layer has its own independent settings.

Setting	Description
Enable	Activates Deep EXR output for this view layer.
Max Depth	Maximum deep layers per pixel (surface + volume). -1 = unlimited on CPU; on GPU -1 maps to 96 — raise it for very dense volumes.
Z Threshold	Minimum Z distance to merge nearby surface fragments (scene units).
Volume Step Size	Raymarch step size for volume slabs (scene units). Match it to your volume scale — e.g. 0.01 m works well for smoke at 1 m scale.
Transparent vs World	How transparent / alpha surfaces interact with the world background in the deep.
Cache Path	Temporary directory for the per-tile deep fragment cache during rendering. Fragments are streamed to disk after each tile to keep RAM usage flat regardless of scene complexity. A local SSD is strongly recommended.
Color Depth	Float Full (32-bit) or Float Half (16-bit) per channel.
Codec	EXR compression for the deep file. Deep OpenEXR only allows ZIPS (default, recommended), RLE, or None.
Override Output Path	Custom output path template (supports // and ##### frame notation).

Setting	Description
Volume DOF/MB Samples	Sub-ray count for Depth of Field and Motion Blur on the volume raymarch (in Sampling > Render). Default 4; raise it for smoother bokeh or motion on the volume slabs, at a proportional cost. Only has an effect when the camera has DOF or motion blur.

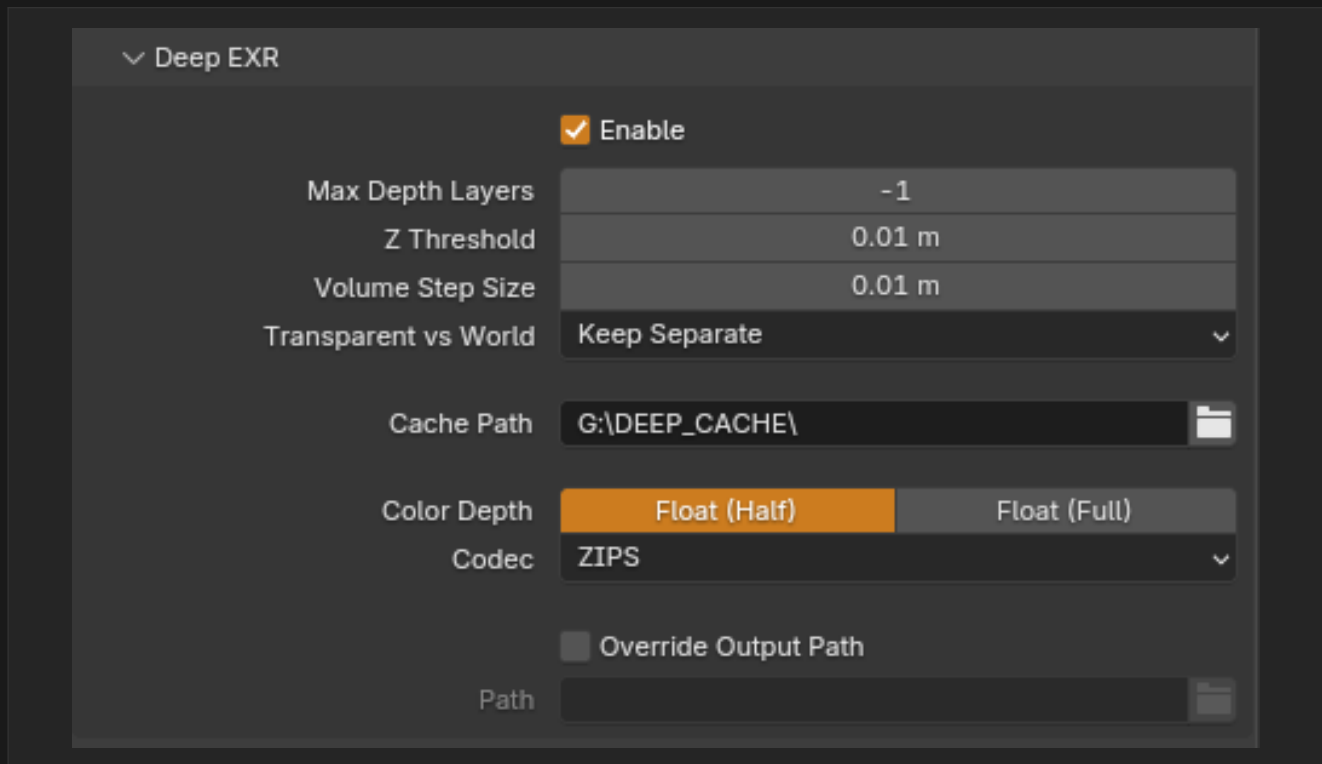


Fig. 1.3 – The Deep EXR panel in View Layer Properties

1.3 Quick Start

Follow these steps to produce your first deep EXR:

- 1 Set the render engine to **Cycles**. For GPU rendering, pick your device (OptiX / CUDA) in **Render Properties > Device**.
- 2 In **View Layer Properties > Passes > Deep EXR**, check **Enable**.
- 3 Set a **Cache Path** (e.g. "C:/DEEP_CACHE/" or "//DEEP_CACHE") — it must be writable; a local SSD is recommended.
- 4 Adjust **Volume Step Size** to match your scene scale.
- 5 Set your output path, or use **Override Output Path** for a custom location.

6 Render — the deep EXR is written after the render completes.

Disable Noise Threshold for deep

Turn **Noise Threshold** (adaptive sampling, in Sampling > Render) **off** when recording deep. Adaptive sampling stops individual pixels at different sample counts, which breaks the deep fragment accumulation — the deep output will be wrong. This is expected; render deep with a fixed sample count (Noise Threshold at 0).

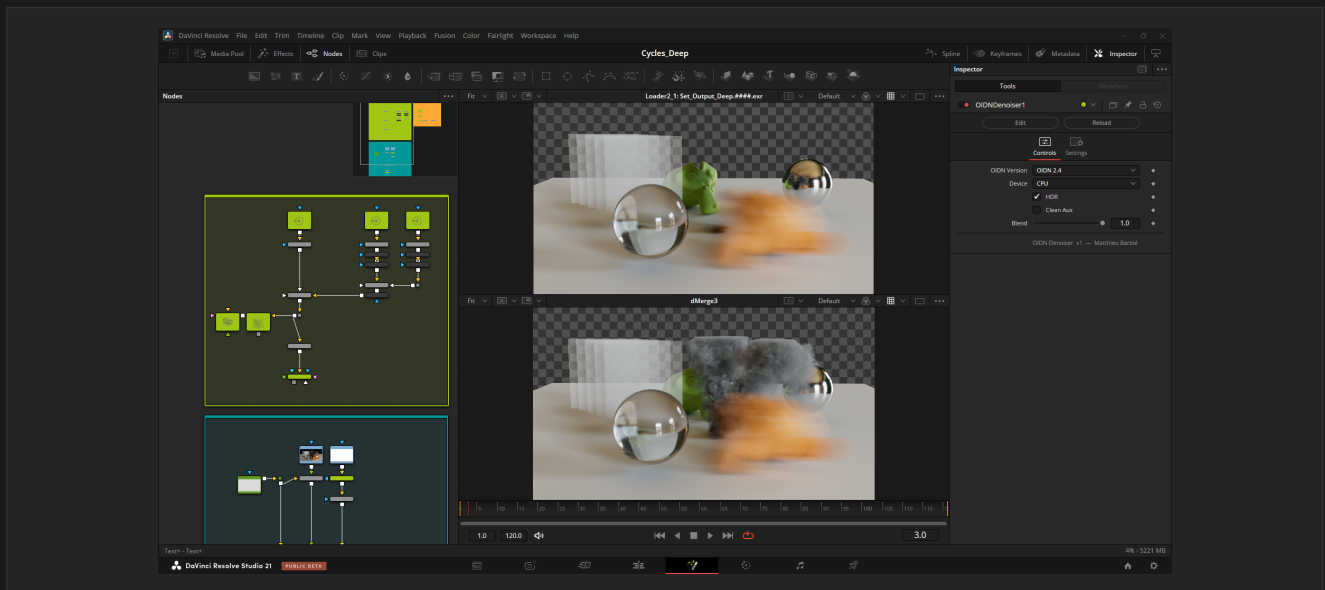


Fig. 1.4 — Basic deep compositing from the resulting deep EXR

1.4 Output Structure

Each view layer is written to its own subfolder with its own filename prefix, so multi-layer setups stay organised automatically:

```
/
Deep/
View_Layer/
View_Layer_output.0001.exr
FX_Layer/
FX_Layer_output.0001.exr
```

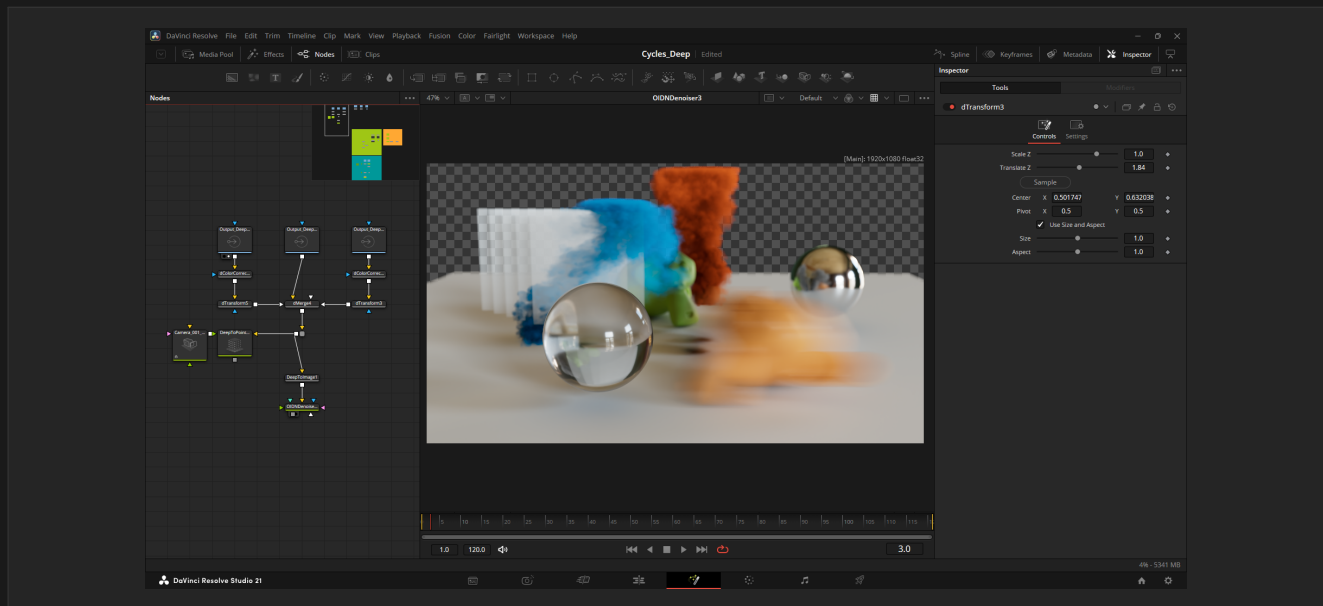


Fig. 1.5 – Volumes at different colours and depths, composited in Fusion

1.5 GPU Rendering & Volume Modes

Deep EXR runs on **CPU**, **CUDA** and **OptiX**, with both SVM and OSL volume shaders on GPU. For GPU rendering, select your device in **Render Properties > Device**.

Biased and Unbiased both supported

Both volume integration modes work — **Unbiased** (delta-tracking) and **Biased** (ray-marching) — with no special setup for the deep.

GPU per-pixel cap

GPU stores deep layers in a fixed per-pixel budget: **Max Depth** defaults to 96 on GPU (the -1 sentinel) and can be raised to 1024, while CPU is unlimited.

The volume march **never truncates**: once the budget would be exceeded it spreads its fragments across the whole span instead of dropping the far ones, so the volume stays complete with thicker slabs. Raise Max Depth when a shot needs finer depth resolution rather than to avoid losing data. If a budget is ever exceeded anyway, the Deep EXR panel raises a red warning and a render report lands in the status bar.

1.6 Multi-Layer Deep EXR Output

A dedicated **output format** that writes every view layer's deep data into a single multi-part OpenEXR — **one deep part per view layer** — instead of one deep file per layer in subfolders. Select it in **Output Properties > Media Type > Multi-Layer Deep EXR**, right next to *Multi-Layer EXR*.

- **One deep part per layer** — each view layer becomes its own deep part (RGBA + Z + ZBack), named after the layer. No per-layer subfolders.

- **A true deep file** — the file contains only deep parts; the beauty is the deep flatten (DeepToImage).
- **Single compression** — one codec (None / ZIPS / RLE) for the whole file, set in the Output panel; these are the only codecs valid for deep data.
- **Per-layer stats** — cycles.<layer>.samples / render_time are written per layer, just like a flat multi-layer EXR.
- **Automatic** — selecting the format enables deep recording for the render.

Compositor compatibility — Fusion / Resolve vs Nuke

Blackmagic Fusion / DaVinci Resolve read every deep part — all layers from one file, a real convenience there. **Nuke** reads only the **first** deep part (its DeepRead handles one part per read); for isolated per-layer deep in Nuke, use the standard output instead, which writes one deep file per layer in Deep/<layer>/.

Deep file vs AOVs

Need the flat passes / AOVs (Diffuse, Normal, Cryptomatte...) too? Render them in parallel as a standard Multi-Layer EXR — the usual VFX split of a deep file and an AOV/beauty render. Choosing any non-deep format keeps the per-layer behaviour from section 1.4 (flat render plus per-layer deep files in Deep/<layer>/ subfolders).

Roadmap — deep AOVs / DeepID

Embedding the passes inside the deep (per-fragment Diffuse, IDs...), and single-file per-layer isolation in Nuke, require per-fragment recording — that is the upcoming DeepID feature. This deep-only format is the foundation it will build on.

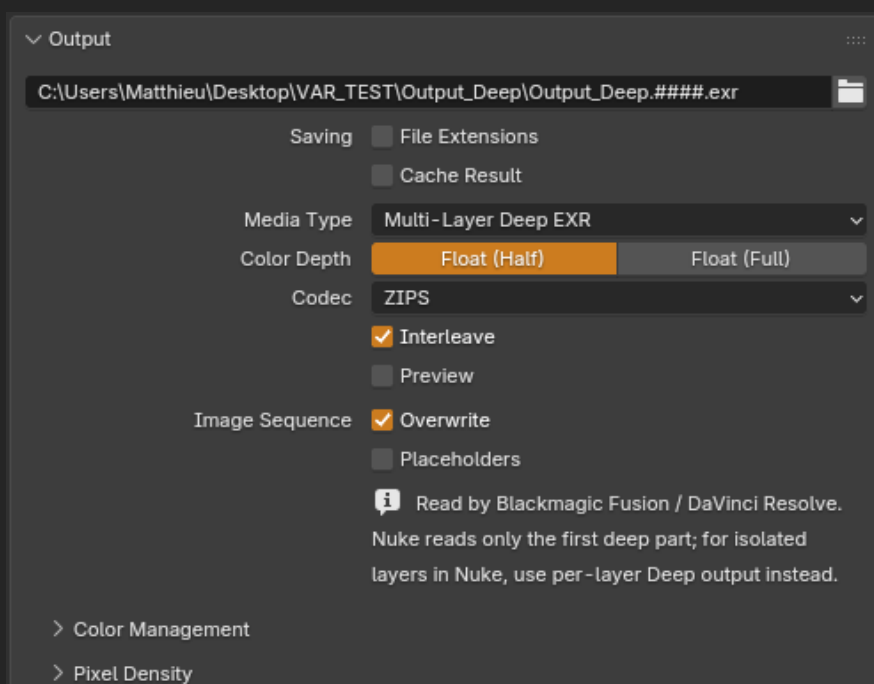


Fig. 1.6 — Selecting Multi-Layer Deep EXR in the Output panel

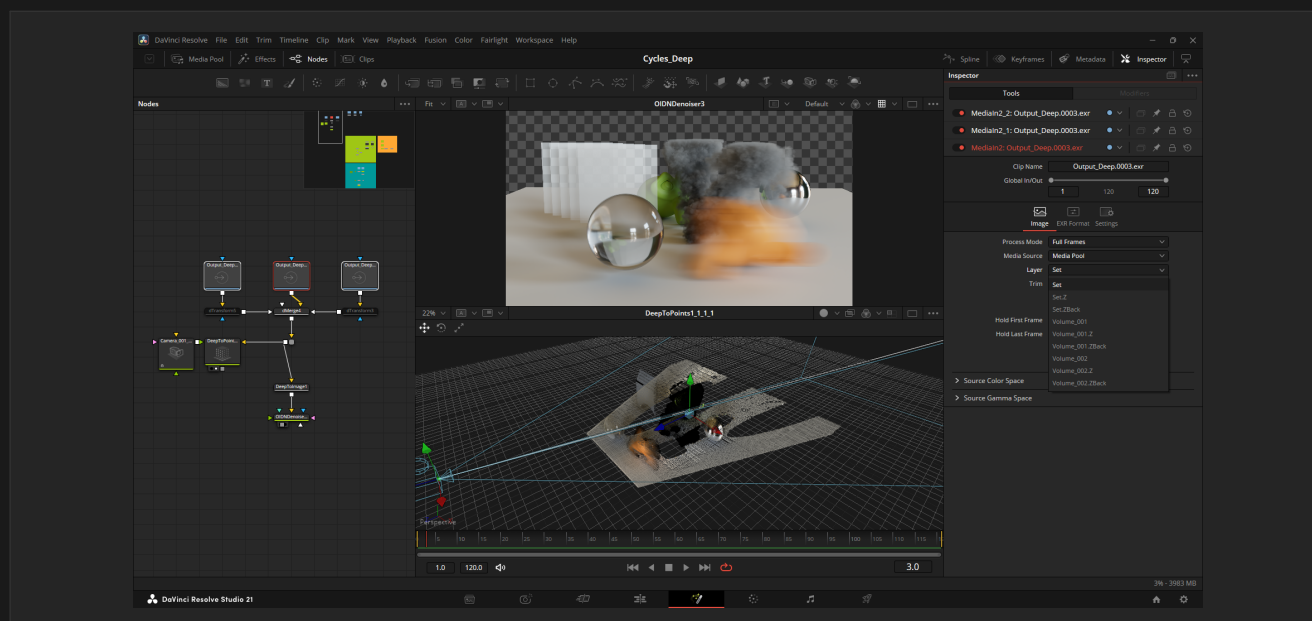


Fig. 1.7 — Multi-Layer Deep EXR read in DaVinci Resolve — Fusion

2

Z-Depth Pass

Antialiased, volume-aware and transparency-aware depth

The standard Z depth pass is upgraded to an **antialiased, volume-aware and transparency-aware** depth. There is no mode selector — just enable **Depth**, and it works with or without Deep EXR.

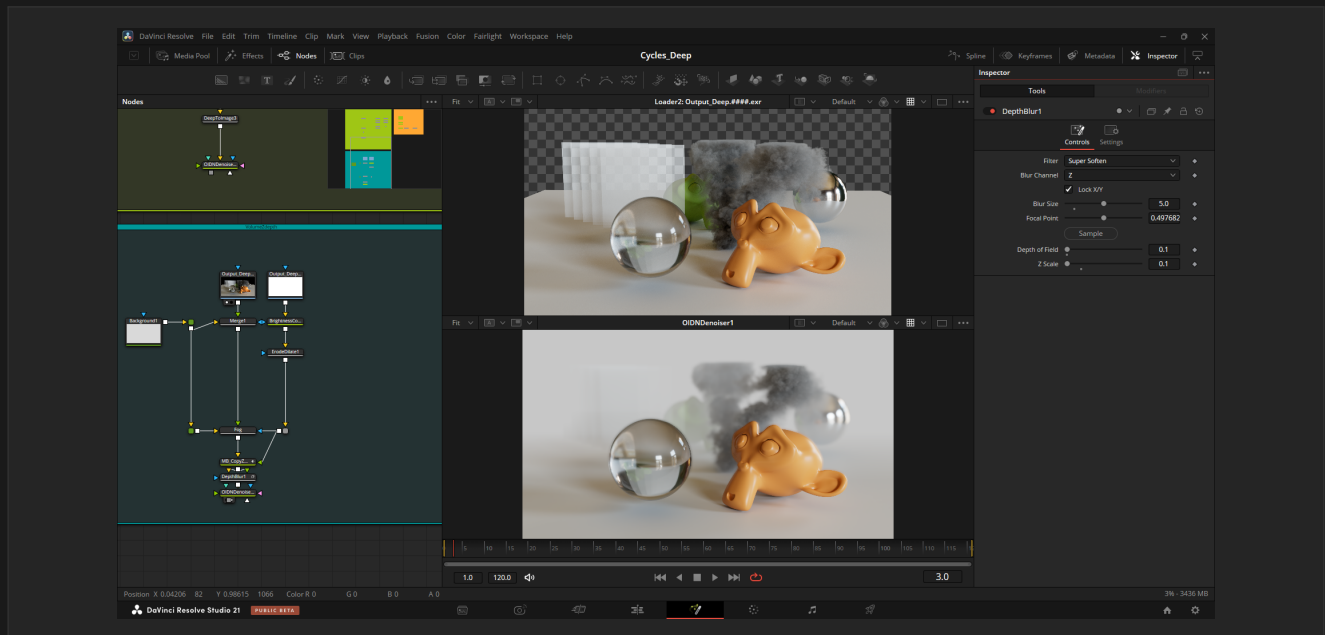


Fig. 2.1 — New Z-Depth with volumes, viewed in DaVinci Resolve — Fusion

2.1 Overview

The final **Depth** channel is a **min(surface, volume)** combine — *nearest wins*, like a compositor "Darken". This is immune to the over-bright that an alpha-over depth produces under motion blur and depth of field.

Difference vs stock Blender

This diverges from stock Blender's Z (non-antialiased, single sample, 1e10 on empty pixels). The output here is a true antialiased depth, ready for Normalize, defocus, and depth-driven atmospherics.

CPU, CUDA & OptiX — SVM and OSL

The antialiased volume-aware Z-Depth runs on CPU, CUDA and OptiX, with both SVM and OSL volume shaders on GPU. On OptiX, OSL volume shading is resolved through the OSL shade pipeline, so the volume depth is computed natively (no surface-only fallback).

Near / Far / Normalize — and what Far really means

Under the **Depth** toggle you set **Near**, **Far** and **Normalize**, per view layer, exactly as you would for the Mist pass. Normalize remaps [Near, Far] into 0–1; raw mode keeps scene units.

Far is more than a display range — it is the declared depth of the background. Anything seen through to the world reads at Far, so a volume floating beyond Far over empty background is pinned to it, while real geometry and volumes composited against it are not clamped in raw mode. Set Far beyond the farthest thing you care about; too large and the whole frame is squeezed into the bottom of the range, reading dark and low-contrast.

Locking the range by hand is also what keeps the pass **stable over an animation**: it no longer references the frame's own maximum depth, so an object entering or leaving the shot cannot shift every semi-transparent pixel.

2.2 What the Pass Does

- **Antialiased surface depth** — accumulated per sample and coverage-weighted, so edges are smooth instead of the hard, aliased single-sample Z of stock Blender. The value is the true (unpremultiplied) scene depth.
- **Volume aware** — volumes contribute a per-pixel depth (jittered sub-rays for clean silhouettes, with Depth of Field and Motion Blur), composited so the nearest of surface / volume wins. Multiple and overlapping volumes are handled correctly.
- **Opacity-weighted volume depth** — a low-opacity volume (wispy smoke, thin atmosphere) is alpha-over composited against the **measured surface depth of that same pixel**, not against a scene-wide constant. A fully opaque volume reads at its true near depth; a nearly transparent one reads close to whatever stands behind it. Where nothing is measured behind, the volume simply keeps its own depth.
- **Transparency aware** — semi-transparent surfaces (and antialiased edges) over the background read at a weighted depth blended toward the background, so a depth-driven fog / atmosphere shows correctly through them.
- **Marched at the volume's own resolution** — the volume depth is marched at each volume object's own **Step Rate**, the same value the render marches with, so the pass resolves whatever the render resolves. There is no extra setting to tune.
- **Locked Near / Far range** — empty pixels are filled with **Far**, a value you set, so a Normalize node maps the whole frame cleanly (no 1e10 spike like stock Blender) and the pass stays stable across an animation.
- **Visible in the viewport** — *Depth* is listed in the viewport Render Pass menu, readable with Normalize enabled.

2.3 Limitations

Motion blur over volumes. A motion-blurred surface partially in front of a volume can produce a hard depth edge at the motion-blur boundary, because the surface and volume depths are combined per pixel after each is resolved.

Workaround — split the Depth only

Only the **Depth** needs splitting, not the beauty. Render the surface and the volume on two separate View Layers, each with its own Depth pass, and combine the **two Depth passes** with a Math > Minimum node. The color / beauty pass can still contain both surface and volume together — it is only the Z-Depth that needs the two-layer treatment. Each Depth is then computed in its own clean context and the motion blur stays correct.

Surface / volume boundary. A thin gap or hard edge may be visible at the contact line between a surface and an adjacent volume. A one-pixel **Erode / Dilate** in the compositor is enough to clean it up in most cases. The two-View-Layer workaround above also eliminates it.

3

GPU Render Options

Direct render-device access and the OSL group-data budget

Two GPU render conveniences are surfaced directly in **Render Properties**: pick your render devices without opening Preferences, and raise the per-shader OSL group-data budget when heavy OSL graphs need it.

3.1 Render Devices in the Device Panel

The Cycles **Device** panel (Render Properties) now shows the render devices directly when GPU is selected — the backend (CUDA / OptiX / HIP / oneAPI) and the per-device checkboxes, mirrored from Preferences. Switch backend or choose which GPUs to render on without leaving the render settings.

3.2 OSL Group-Data Budget

Open Shading Language shaders allocate a fixed per-thread *group-data* buffer on GPU, capped at **2048 bytes** in stock Blender — heavy custom OSL graphs can exceed it and fail to load with a "group data size" error. This build adds an **OSL GPU Group Data** selector in **Render Properties** (shown when **Open Shading Language** is enabled): from **2048** (default) up to **6144** bytes, in 1024-byte steps. Raising it lets heavier custom OSL graphs compile and render on GPU.

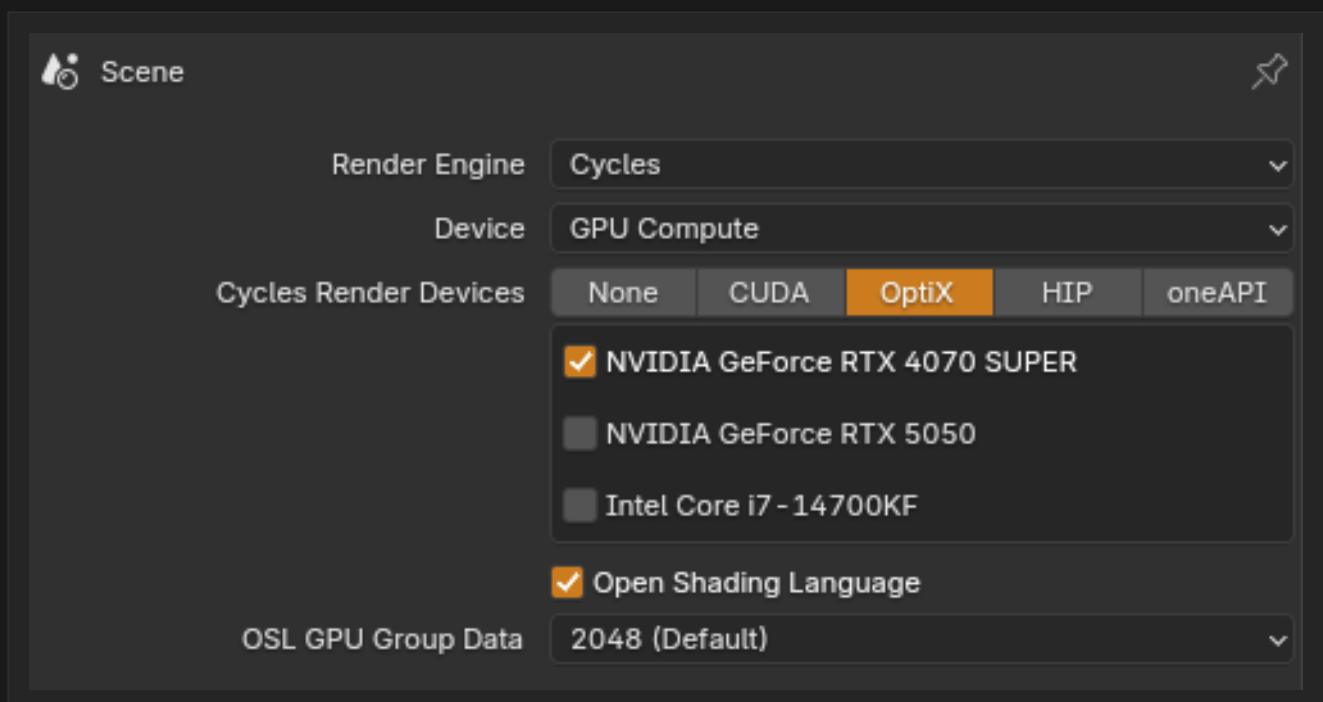


Fig. 3.1 — Render devices and the OSL group-data budget in Render Properties

3.3 Trade-off & Constraints

Trade-off

The buffer is reserved per-thread, so a larger budget lowers GPU occupancy and slows down all OSL shading in the render — the impact can be significant. It only affects GPU rendering; OSL on CPU is unlimited. Changing the value recompiles the OSL kernels.

If a shader still exceeds the highest budget, simplify the OSL graph (fewer layers, break up node groups, drop large arrays) or render that shader on CPU, where group data is unlimited. The **CYCLES_OSL_GROUPDATA_ALLOC** environment variable overrides the menu for advanced setups.

3.4 Choosing a Backend

All build features are validated on **CPU**, **CUDA** and **OptiX**, and in multi-device (**XPU**, CPU + GPU together). Which backend is best depends on the scene:

Workload	Recommended backend
Volumes, Deep EXR, XPU	CUDA — the reference path for these features. On volume-heavy scenes it matches OptiX within a few percent (the raymarch is not RT-core work), and CPU + CUDA stay perfectly consistent in multi-device.
Heavy geometry, hair / fur	OptiX — hardware BVH traversal (RT cores) and native curve primitives pay off on millions of triangles, dense instances and fur.
OSL shaders on GPU	OptiX — required. GPU OSL shading is OptiX-only; CUDA is SVM only.

XPU note

*Multi-device (XPU) is fully supported for Deep EXR and every pass, on both CUDA and OptiX. One edge case remains on **XPU + OptiX** only: a camera-visible volume in an Indirect-Only collection sitting over a Shadow Catcher can show a faint seam in the Shadow Catcher pass at the CPU / GPU tile boundary (the OptiX 8-bit trace mask can't match the CPU's volume visibility). It does not affect CUDA XPU, pure OptiX, or any other pass — render such shots on CUDA, or set the volume camera-invisible.*

4

Environment Variables in File Paths

Portable, pipeline-friendly path resolution

File paths anywhere in Blender — render output, textures, libraries, caches, and even Preferences — now support **environment variable expansion**. This makes .blend files portable across machines and render farms with different local configurations.

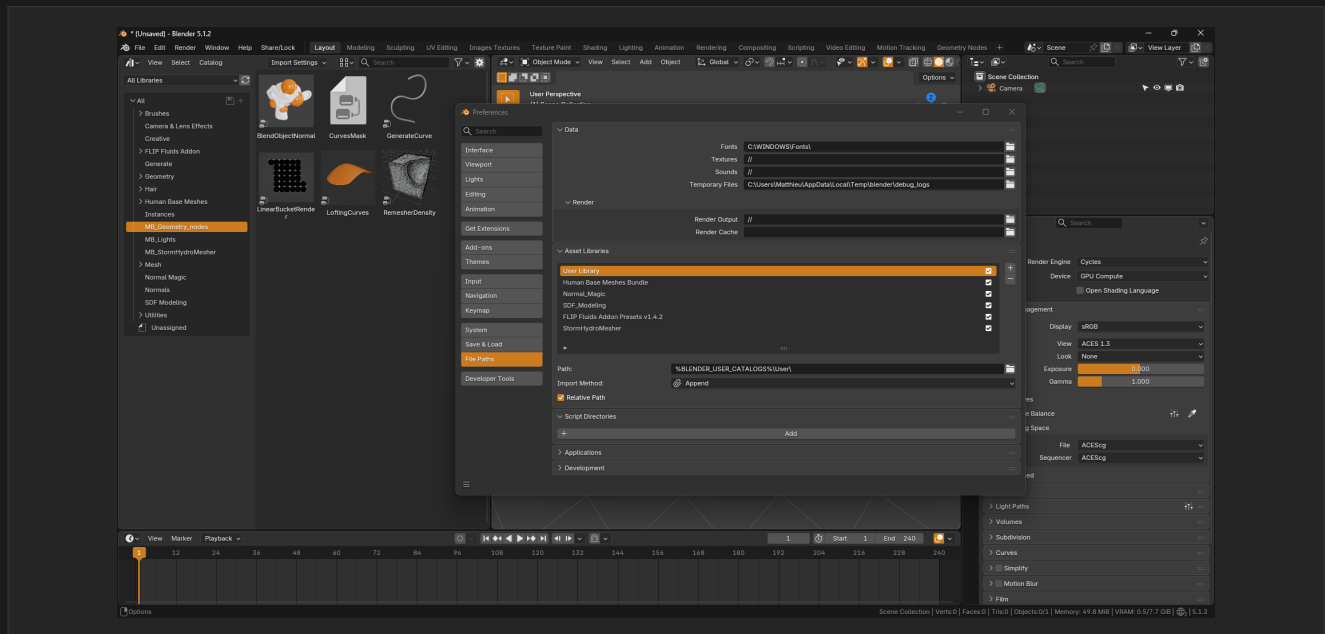


Fig. 4.1 — Full environment-variable support across Blender file paths

4.1 Overview & Supported Syntaxes

All three common syntaxes are supported, cross-platform:

```
$MY_PROJECT/renders/####.exr
${SHOT_DIR}/textures/diffuse.png
%PIPELINE_ROOT%/assets/char.blend
```

Variables are expanded at **path resolution time**, not at load time, so a .blend file stays portable: the same scene resolves to the right local paths on each artist's machine and on the farm, driven entirely by the environment.

Tip

Combine environment variables with Blender's // relative-path notation and #### frame padding for fully pipeline-driven, machine-independent output paths.

5

Qt Tools (PySide6)

An extensible, separate-process Qt UI framework for pipeline tools

This build bundles a small framework to run rich **Qt (PySide6)** user interfaces for pipeline tools — asset browsers, farm submitters, advanced control panels — beyond what the native Blender UI allows.

5.1 Overview

- **Separate process** — each Qt window runs in its own process, fully isolated from Blender. Dragging or resizing the Blender window never freezes the tool (and vice-versa).
- **Always on top** — tool windows stay above Blender so they are never lost behind it.
- **Live two-way sync** — editing a value in the tool updates Blender instantly; changes made in Blender flow back to the tool.
- **Auto-loaded bridge** — the headless **MattRM2 Qt Bridge** add-on loads automatically; it relays data between Blender and the tools and has no UI of its own.

5.2 Enabling the Deep EXR Demo

A Deep EXR control panel ships as a demo of the framework. To use it:

- 1 In **Preferences > Add-ons**, enable **MattRM2 — Deep EXR Panel (Qt demo)** (search "MattRM2").
- 2 In the 3D Viewport, open the **N-panel** and go to the **MattRM2 Qt** tab.
- 3 Click **Open Deep EXR Panel** — the Qt window opens, live-bound to the Deep EXR render settings.

5.3 Building a Tool — for Developers

A control-panel tool is essentially a layout of Blender property paths. The shared SDK builds the right widget per type and handles all the IPC and theming:

```
from mattrm2_qt_sdk import run_control_panel

run_control_panel("My Tool", layout=[
    ("Output", [
        ("Codec", "scene.render.deep_exr_output_compression"),
    ]),
])
```

Architecture:

- **Bridge** (auto-loaded, Blender side) — a stable public API: `launch_tool()`, `register_handler()`, `register_tools_dir()`. It resolves context-relative RNA paths (e.g. `scene.render.codec`) and relays get/set.
- **SDK** (Qt side) — the shared charter: Matt Dark theme, always-on-top window, auto-built widgets from the schema, two-way sync. Tools import it.
- **Where code lives** — UI and third-party integrations (render farm, asset / production trackers) live in the Qt tool process (its own Python, free to add dependencies); custom Blender-side logic is registered as a handler on the bridge.

License

Qt for Python (PySide6) and Qt are bundled under the GNU LGPL v3 (dynamically linked). The license texts are in the distribution's `licenses/` folder.

6

View Layer Attribute Overrides

Maya-style per-render-layer overrides — up to the render engine

Override almost **any property per view layer** — the workflow of Maya's Render Setup, native in Blender. Right-click a property, choose **Add Layer Override**, and that layer gets its own value; every other layer keeps the base one. The system is a dedicated per-layer store (separate from Library Overrides), applied on each layer's evaluated data — the original scene data is never touched.

6.1 Overview & Workflow

- **Add / Remove** — right-click any supported property: object and light/camera data attributes, shader node sockets, and render settings. *Add Layer Override* captures the current value for the active view layer.
- **Visual state** — overridden properties show a **bold orange label** (color configurable in Preferences > Themes > User Interface > State); checkboxes highlight their own text.
- **Per-layer editing** — with an override active, editing the property (fields, color picker, eyedropper, copy/paste, hex) changes **only the active layer**. Switch layers and the UI, tooltips and material preview follow the layer's values.
- **Persistent** — overrides are saved in the .blend per view layer and restored on load.

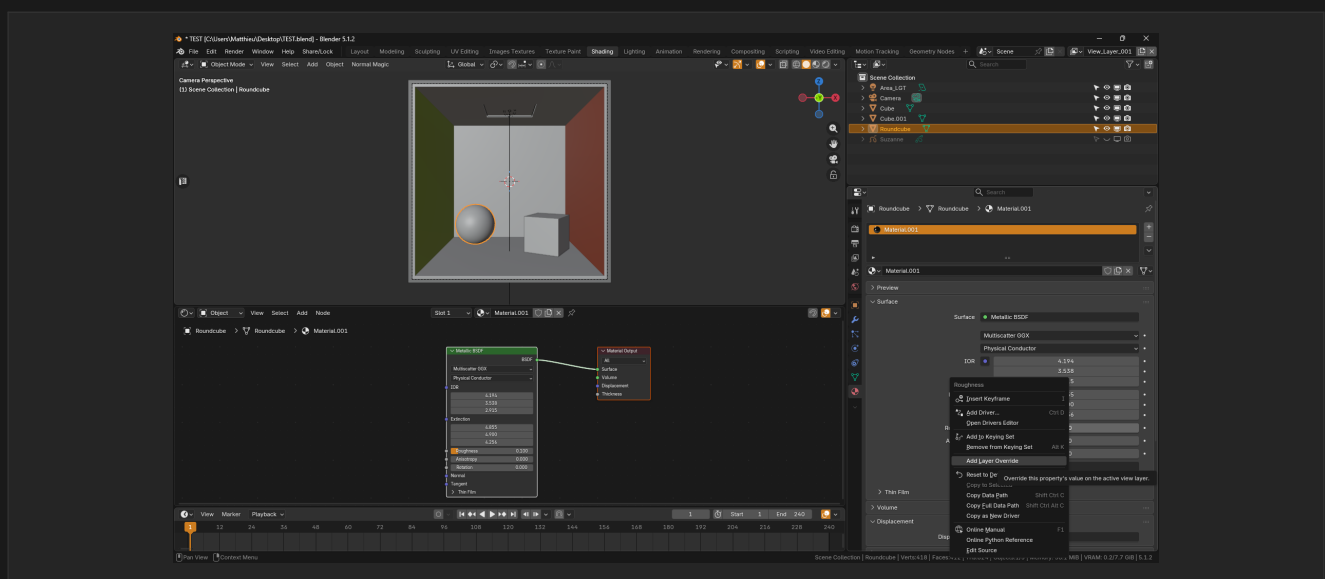
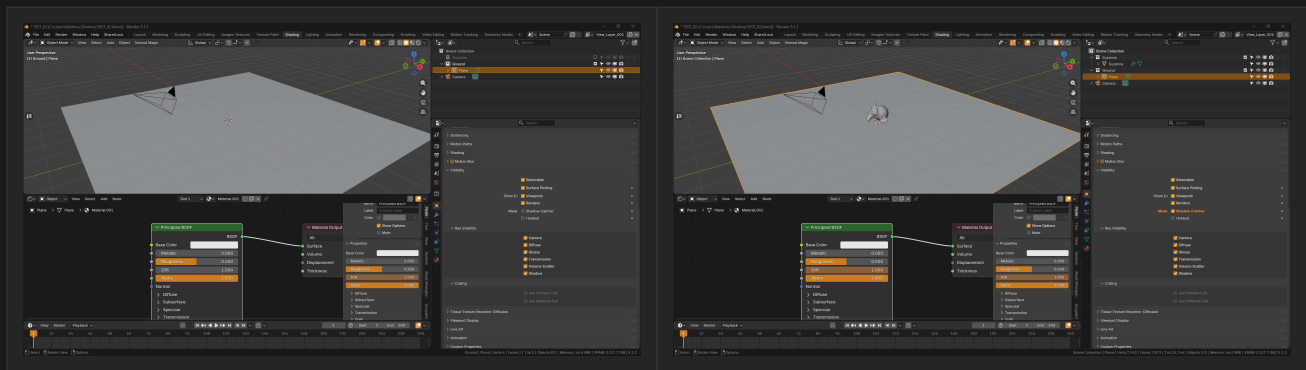


Fig. 6.1 — Right-click a property: Add / Remove Layer Override



Before — base properties, shared by all layers

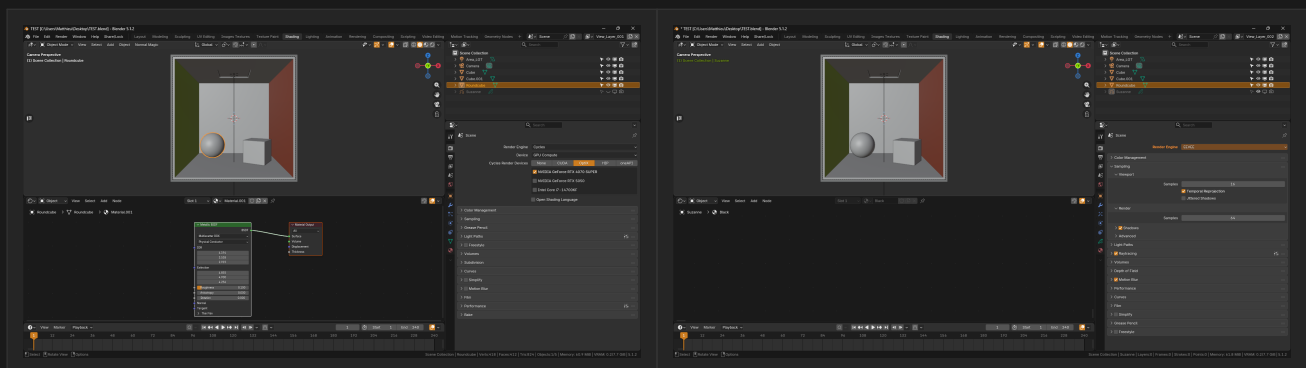
After — per-layer overrides active: bold orange labels, layer-local values

Fig. 6.2 — Overridden properties: bold orange labels, per-layer values.

6.2 Per-Layer Render Engine

The render engine itself can be overridden per view layer: one scene can render some layers with **Cycles** and others with **EEVEE** in the same F12 or animation render, into the same multilayer EXR — each layer with its own engine's pass layout.

- **Right-click the Render Engine dropdown** on the target layer and add the override, then pick the engine for that layer.
- **The whole UI follows** — render panels, view layer passes and shader node availability switch to the active layer's engine; the rendered viewport previews the active layer with its own engine.
- **Grease Pencil and Freestyle** keep compositing correctly on mixed-engine renders.



View Layer 01 — scene engine (Cycles)

View Layer 02 — engine overridden to EEVEE: render UI and viewport follow

Fig. 6.3 — Render engine per view layer: Cycles and EEVEE in one scene.

6.3 Per-Layer Samples & Shaders

- **Samples** — override *Max Samples* per layer (e.g. 4096 on the hero layer, 64 on a matte layer): each layer renders with its own count in the same render, wired into Cycles' native per-layer sampling.
- **Shader switching** — override any node socket. Overriding a *Mix Shader* factor turns it into a per-layer shader switch without duplicating materials; the material preview shows the active layer's look.

- **Engine settings** — Cycles and Eevee settings read per layer (bounces, clamps, film transparency, motion blur toggles...), so specialized layers carry their own render configuration.
- **Output file** — override *Output > Output Path* and that layer is written to its own file sequence. Override the *Media Type / File Format* block too, or simply type the extension you want: a PNG layer and an EXR layer can come out of the same render. When every rendered layer has its own output, no combined file is written.
- **Frame range** — override *Frame Start / Frame End* per layer: render a static matte once while the animated layer runs the full range, in a single job. Read the warning below before using it.

6.4 Scope & Limitations

Frame range overrides restrict F12 too

A FRAME RANGE OVERRIDE ALSO APPLIES INSIDE BLENDER, IN F12. *This is the trade-off of the frame range override, and it is not a bug. Once a view layer carries a **Frame Start / Frame End** override, **only the frames it covers are ever rendered** — including a plain F12 in the interface, not just command-line or animation renders.*

*On a frame outside that range the layer is **not computed at all**: it is absent from the render result, nothing is written for it, and the Render Result window shows nothing when that layer is the active one. A layer's range is also **intersected** with the range you ask for, so it can never extend a render beyond it.*

To get a layer back on every frame, remove its frame range override.

Pipeline-geometry settings — **resolution, borders and FPS** — remain deliberately not overridable: they are read once per render from the pipeline's own copy of the render settings, and keeping them per scene guarantees a consistent multilayer EXR. The right-click entry is disabled for them. Animated or driven properties cannot be overridden either (the override and the F-Curve would fight over the value).

Per-layer outputs

*A per-layer output cannot produce a video file — a stream cannot be split per layer, so the override is ignored for movie formats. **Multi-Layer Deep EXR** does follow the override: each layer is written as its own single-part deep file at its own path, as long as that path ends in **.exr** — point a layer at a format that cannot carry deep (PNG, JPEG...) and it is written flat, with a warning in the report. The standard **Deep EXR** output is unaffected and keeps its own **Deep/<layer>/** naming.*

Per-layer output vs. the compositor

Do not combine a per-layer output override with Blender's compositor in the same scene. *Per-layer files are written straight from the render layers, **before compositing** — which is exactly what you want when you finish in Nuke, Fusion or Resolve, and why the feature exists. The combined file, on the other hand, is the compositor's own output (standard Blender behaviour), so a layer left without an output override loses its own image to the composite.*

*Rule of thumb: **no output override** — drive your outputs from the compositor, as usual. **Output override** — keep the compositor out of that scene. A pipeline that serves both is on the roadmap.*

7

Curves Lofting

A real loft node for Geometry Nodes

Curves Lofting sweeps a surface through a series of curves that share the same number of evaluated points — the loft of Maya, Houdini and Rhino, as a node instead of a node-group workaround built on a grid. Find it under **Add Curve Operations**, next to *Curve to Mesh*.

7.1 Inputs and Options

The node starts with two curve inputs and grows: drop a link on the dotted socket, or press **Add Item** in the sidebar, and the inputs are numbered for you. Removing one rennumbers the rest and leaves the other links in place. An input holding several curves adds them all, in index order — so a single input carrying five curves lofts them just as five separate inputs would.

Input	What it does
Collection	A whole collection of curve objects, lofted before the numbered inputs. Sorted by name — see 7.2.
Interpolation	Linear — straight from one curve to the next. Spline — a Catmull-Rom that passes through every curve.
Spline Type	Shown in Spline mode. Uniform weights every curve equally; Centripetal accounts for the spacing and avoids the bulges that unevenly spaced curves produce.
Subdivisions	Intermediate sections created between each pair of curves. 0 by default, where Linear and Spline give the same result.
Close Loft	Joins the last curve back to the first, for a ring or a torus. Needs at least three curves.

7.2 Lofting a Collection

The **Collection** input takes a whole collection of curve objects, lofted before the numbered inputs. Since a collection has no order of its own, **the objects are sorted by name** — which makes the naming your ordering tool:

Curve.001 Curve.002 Curve.003 (Blender's own numbering)
 Curve_001 Curve_002 Curve_003 (an explicit numbering)

The comparison is **natural**, so **Curve.2** comes before **Curve.10** rather than after it, and a number does not need padding to sort correctly. Collections nested inside the collection are included, and their objects take part in the same global sort. Objects that are not curves — meshes, lights, empties — are ignored, so a working collection does not have to be cleaned up first.

Transforms

Each object's transform is applied, so the curves loft where they sit in the scene. Feeding the node a Collection Info node instead keeps working, as long as its instances are realized first – the Collection input saves you that step, and sorts.

7.3 Outputs

Three outputs, all built from the same interpolation:

- **Mesh** – the lofted surface, quads all the way.
- **UV Map** – a field: U runs along the curves, V from the first curve to the last. Store it with *Store Named Attribute*.
- **Curves** – the input curves plus every section generated between them, as poly curves, ready to be re-swept or re-sampled.

Point attributes are interpolated along with the surface, so an attribute captured on the input curves survives the loft. Per-curve attributes, **material_index** included, stay constant across each band of faces, which is what you want for materials.

7.4 Rules

Same point count, same direction

*Every curve needs the same number of **evaluated** points and the same direction. A Resample Curve before the loft settles the first, a Reverse Curve the second – the node tells you which rule was broken, and with which counts. Closed and open curves cannot be mixed in one loft: make them all closed, or all open.*

Two curves are always straight

With only two curves the surface is straight whatever the interpolation, exactly as in Maya and Houdini: two sections carry no information about how the surface should bend between them. Spline starts to show from three curves on.

8

Included Bug Fixes

Upstream Cycles / UI fixes shipped ahead of official patches

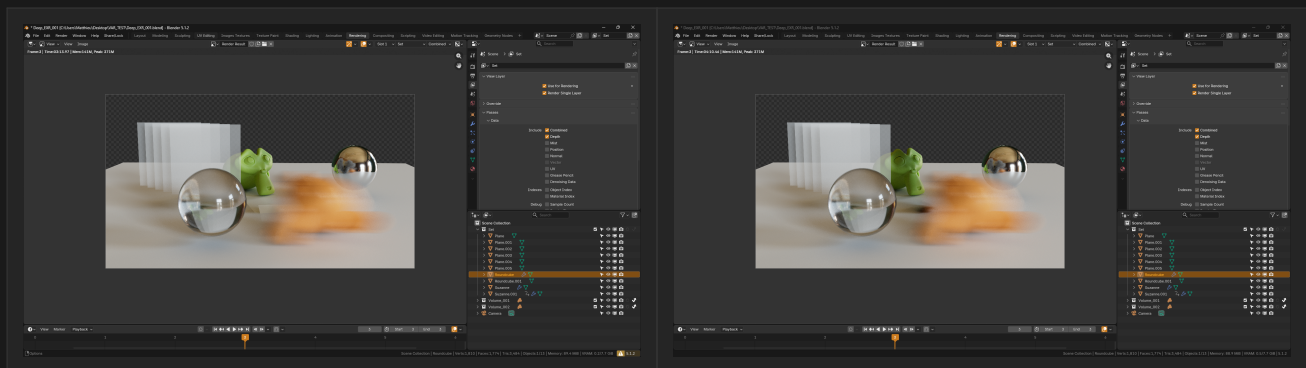
8.1 Node Editor Click-Drag

Fixed a Blender bug (still present in official 5.2.2) where click-dragging a node would move a different node than the one under the cursor. The incorrect node selection persisted in the .blend file. This fix is included ahead of the upstream patch.

8.2 Volume Indirect-Only Shadows

Fixed a Cycles bug where volumes placed in **Indirect-Only** collections would not cast shadows — on surrounding surfaces, and on other volumes (volume-on-volume).

In particular, when the volume's bounding box intersected the floor or another object, the shadow disappeared on the intersecting surface. This build renders the shadow correctly even with large bounding-box intersections, on **CPU**, **CUDA** and **OptiX**.



Before — stock Blender 5.2.2: no shadow where the volume's bounding box meets the floor

After — this build: shadow cast correctly, even on large bounding-box intersections

Fig. 8.1 — Volume Indirect-Only shadow, before / after the fix.

Note

The underlying issue is still present in official Blender 5.2.2 and will be reported upstream. This build ships the fix already applied.

8.3 Texture Cache with CPU + GPU Rendering

Fixed a Cycles regression where enabling the **Texture Cache** while rendering on CPU and GPU together produced large black regions and corrupted textures. Rendering was correct on a single GPU, on two GPUs, or on CPU alone — only the mix failed.

The multi-device wrapper reported unified image memory as soon as *any* single device had it, and the CPU always does. Cycles concluded there was nothing to upload, so the GPUs rendered their share of the frame without ever receiving the texture tiles.

Note

Upstream fix, merged for 5.3 and tagged for backport to 5.2 but not present in 5.2.2. It is applied here ahead of the official patch.

8.4 Blend File Icon on Windows

Fixed the Windows file association writing its icon reference as a positional index rather than a resource ID, which made Explorer display the **application** icon on .blend files instead of the document icon. The registration now names the icon resource explicitly, which is unambiguous across Windows versions.

The same defect is present in official Blender, where it goes unnoticed because both icons carry the same mark.

9

Known Limitations & Roadmap

Current constraints and what's coming next

9.1

Known Limitations

Limitation	Notes / Workaround
Deep EXR is incompatible with Noise Threshold (adaptive sampling)	Adaptive sampling stops pixels at different sample counts, which breaks deep fragment accumulation. Set Noise Threshold to 0 (off) and render deep with a fixed sample count. Expected behaviour, not a bug.
GPU deep uses a per-pixel layer cap	CPU is unlimited. Max Depth -1 maps to 96 on GPU and can be raised to 1024. The march coarsens rather than truncating, so the volume stays complete; raise it in the Deep EXR panel for very dense volumes.
Multi-Layer Deep EXR: Nuke reads only the first deep part	It is a Fusion / Resolve format. For Nuke, use the standard output — one deep file per layer in Deep/<layer>/ — read individually with full isolation.
View Layer Overrides: pipeline settings are locked	Resolution, borders, frame range, FPS and output paths/formats cannot be overridden per layer (read once per render). Use the compositor File Output node for per-layer outputs — native support is on the roadmap.
View Layer Overrides: animated / driven properties	Cannot be overridden: the override and the F-Curve would fight over the value. Remove the animation first.
Heavy OSL graphs may exceed the GPU group-data budget	Raise the OSL GPU Group Data budget in Render Properties (up to 6144), or override it further with the CYCLES_OSL_GROUPDATA_ALLOC environment variable; otherwise simplify the shader, or render it on CPU (unlimited).
Z-Depth: motion blur over volumes	Hard depth edge where a motion-blurred surface is partially in front of a volume. Split only the Depth: render surface and volume on separate View Layers, each with a Depth pass, and min() the two depths in compositing. The color / beauty pass can keep both surface and volume together.
Multi-Layer Deep EXR + multi-view	Cycles deep is mono (single view). The combined deep output is not supported with stereo / multi-view — render mono, or use the per-layer deep files.

9.2

Roadmap

Planned by version. Order and scope may shift as development progresses.

Version	Planned
v1.2	Deep compositing node — a native Blender compositing node for deep data, and deep output support in the Compositing EXR writer .

Version	Planned
v1.3	DeepID — per-fragment objectId, materialId, normal and albedo, with a matching DeepID compositing node .
v1.4	LPE (Light Path Expressions) — custom AOVs via light path expressions (Arnold / RenderMan parity).
v1.5	Advanced caustics — a fast, accurate caustics engine, tracing through volumes.
v1.6	USD stage — Universal Scene Description stage support.
v1.7	Scene management — a Gaffer / Katana-style manager for the USD stage, Alembic and Blender data (in design).

Feedback

This build is in active development. If you encounter unexpected behaviour, crashes, or rendering differences vs. stock Blender, please report with a .blend or minimal reproduction scene, your render settings, and your OS / GPU. Your feedback directly influences what gets built next. — Matthieu Barbié

License & Trademark

MattRM2 VFX Build is an independent build based on Blender 5.2.2 LTS, distributed under the GNU GPL v3. It also bundles Qt for Python (PySide6) and Qt under the GNU LGPL v3 (dynamically linked); the license texts are in the distribution's licenses/ folder. Not created, sponsored, or endorsed by the Blender Foundation. "Blender" is a trademark of the Blender Foundation. — blender.org